

Real-Time Optimization Model

Mathematical Formulation

DAMO 699 Capstone · Group 3
Multi-Tenant Cluster Memory Scheduling

Contents

Terminology	2
1 Inputs to Optimizer	2
2 Decision Variable	2
3 Sets	3
4 Parameters	3
5 Feedback Loops	5
6 Optimization Objective	6
7 Constraints	6
8 Complete Model	7
A How $\bar{v}_n^{\text{SLA}}$ is Computed	8
B How $\omega_n^{\text{utilize}}$ is Computed	9
C How ω_t^{delay} is Computed	9

Terminology

A **job** refers to any containerized workload unit submitted by a tenant for execution on the cluster. In Kubernetes this corresponds to a **pod** — the smallest schedulable unit. We use *job* for generality, as the model is scheduler-agnostic and not specific to Kubernetes.

1 Inputs to Optimizer

The real-time model is stateless: it has no concept of tenants, plan-ahead assignments, or machine ownership. The **Cluster Manager** handles all filtering before the solver is called. Each scheduling round, the Cluster Manager:

1. Filters the job queue to jobs belonging to the current tenant group.
2. Filters the node list to machines assigned to that group by the plan-ahead model.
3. Sorts the filtered jobs by `arrival_round` (FIFO — oldest first).
4. Calls `solve(group_jobs, group_nodes, W_t, K)`.

Cluster monitor inputs (passed per solve call):

- U_n^{mem} — currently used memory per node
- $\bar{v}_n^{\text{SLA}}$ — rolling SLA violation rate per node (last K rounds)
- ω_j^{delay} — scheduling delay weight per job (inherited from tenant; computed over last K rounds)

Prediction layer inputs:

- $\hat{P}_j^{\text{mem}}$ — maximum predicted memory per pending job over its lifetime
- $\hat{P}_j^{\text{CPU}}$ — P95 predicted CPU peak per pending job over its lifetime

The optimizer maximizes Z (Placement Score) and outputs x_{jn} , which the cluster monitor uses to submit jobs to nodes. Plan-ahead access control is enforced by the Cluster Manager’s pre-filtering step, not by a constraint inside the LP.

2 Decision Variable

$$x_{jn} \in \{0, 1\}$$

$x_{jn} = 1$ if job j is assigned to node n ; $x_{jn} = 0$ otherwise. This placement matrix is the sole output of the optimizer.

3 Sets

Symbol	Definition
J	Pending jobs for this solve call — pre-filtered by the Cluster Manager to a single tenant group, sorted FIFO by <code>arrival_round</code>
N	Available nodes for this solve call — pre-filtered by the Cluster Manager to machines assigned to the current tenant group by the plan-ahead model
T	All tenants sharing the cluster (used only for $\bar{W}_t$ computation; not an input to the LP directly)
J_t	Jobs belonging to tenant t , where $J_t \subseteq J$

4 Parameters

Node Limitations

Physical node properties — fixed facts about the hardware.

Symbol	Meaning
M_n	Physical RAM installed on node n
C_n	Total CPU cores on node n

Configuration

Set by the cluster operator before the simulation; constant at runtime.

Symbol	Meaning
M_n^{tax}	Memory tax — fixed overhead reserved for the OS, kubelet, and monitoring agents; not available to tenant jobs. Based on Google cluster-usage traces v3: the Borglet agent reserves $\approx 5\%$ of M_n , making the effective usable ceiling $\approx 95\%$ of physical RAM
M_n^θ	Memory safety threshold buffer — fixed memory reserved per node as protection against runtime memory spikes. Default: $M_n^\theta = 0.10 \cdot M_n$ (10% of physical RAM). Together with M_n^{tax} , defines the hard capacity ceiling M_n^{cap}
K	Rolling window length — number of past scheduling rounds used to compute both $\bar{v}_n^{\text{SLA}}$ and ω_t^{delay} . See Appendices A and C
$\sigma_n^{\text{consolid}}$	Fixed node consolidation weight — static bias that steers placement toward lower-indexed nodes, keeping the cluster consolidated. Node 0 gets the highest weight; the last node gets 1. Constant across all scheduling rounds. $\sigma_n^{\text{consolid}} = N - n$, $n = 0, 1, \dots, N - 1$ <i>Note:</i> A sequencing constraint (node n may only receive jobs if node $n-1$ is non-empty) would create a recursive dependency the MILP cannot resolve in a single solve call. The fixed weight achieves the same consolidation bias without constraint coupling.

Cluster Monitor Inputs

Provided to the optimizer at the start of each scheduling round.

Symbol	Meaning
U_n^{mem}	Currently used memory on node n , measured in real time. Subscript <i>mem</i> distinguishes from future CPU usage metrics (e.g. U_n^{cpu})
$\bar{v}_n^{\text{SLA}}$	Rolling SLA violation rate on node n — fraction of the last K rounds in which $U_n^{\text{mem}} > M_n^{\text{cap}}$. $\bar{v}_n^{\text{SLA}} \in [0, 1]$. $\bar{v}_n^{\text{SLA}} = 0$: no recent violations. $\bar{v}_n^{\text{SLA}} = 1$: every recent round was overloaded. See Appendix A
ω_j^{delay}	Scheduling delay weight for job j — inherited from the tenant that owns job j ($\omega_j^{\text{delay}} = \omega_{t(j)}^{\text{delay}}$); computed over the last K scheduling rounds. $\omega_j^{\text{delay}} \geq 1$. $\omega_j^{\text{delay}} = 1$: no extra weight (tenant wait at or below cluster average). $\omega_j^{\text{delay}} > 1$: additional weight; solver increases preference for job j (real-valued, no maximum). See Appendix C

Prediction Layer Inputs

Provided by the prediction layer for each pending job.

Symbol	Meaning
$\hat{P}_j^{\text{mem}}$	Maximum predicted memory for job j — worst-case peak memory usage predicted over the job’s expected lifetime $\hat{P}_j^{\text{life}}$. Based on Google cluster-usage traces v3 <code>maximum_usage</code> field. Used in constraint C2 and the objective
$\hat{P}_j^{\text{CPU}}$	P95 predicted CPU peak for job j — 95th percentile of CPU demand predicted over the job’s expected lifetime $\hat{P}_j^{\text{life}}$. Based on <code>tail_cpu_usage_distribution</code> (index 4) from Google cluster-usage traces v3. Used in constraint C4. No SLA feedback applies to CPU; kernel throttling handles runtime overrun
$\hat{P}_j^{\text{life}}$	Predicted job lifetime — time horizon used by the prediction layer to compute $\hat{P}_j^{\text{mem}}$ and $\hat{P}_j^{\text{CPU}}$. Not used directly in the LP

Derived

Computed by the optimizer from the above inputs each scheduling round.

Symbol	Meaning
M_n^{cap}	Node capacity — physical RAM minus OS tax and safety threshold buffer; fixed ceiling for all tenant job memory on node n
M_n^{avail}	Remaining available memory — capacity minus current usage; may be negative when over-committed
M_n^{eff}	Effective remaining memory — capacity offered to new jobs after SLA violation scaling. Right-hand side of constraint C2
$\omega_n^{\text{utilize}}$	Memory utilization weight — ratio of current used memory to node capacity M_n^{cap} , mapped to $[1, 2]$. Applied in the objective to consolidate workloads onto busier nodes. See Appendix B

$$\begin{aligned}
M_n^{\text{cap}} &= M_n - M_n^{\text{tax}} - M_n^{\theta} \\
M_n^{\text{avail}} &= M_n^{\text{cap}} - U_n^{\text{mem}} \\
M_n^{\text{eff}} &= \max\left(0, M_n^{\text{avail}} \cdot (1 - \bar{v}_n^{\text{SLA}})\right)
\end{aligned}$$

5 Feedback Loops

SLA and fairness are not objectives inside the LP — they are observed outcomes tracked by the cluster monitor and fed back as adaptive inputs each scheduling round.

SLA Feedback $\rightarrow$ Node Capacity. When actual memory on node n exceeds M_n^{cap} at runtime, the cluster monitor records one overload event. It computes $\bar{v}_n^{\text{SLA}}$ over the last K rounds and passes

it to the optimizer. As violations persist, $\bar{v}_n^{\text{SLA}}$ rises and M_n^{eff} shrinks — the node accepts fewer new jobs, reducing the risk of OOM kills. As violations subside, M_n^{eff} recovers.

Fairness Feedback $\rightarrow$ Job Priority. The cluster monitor tracks per-job wait times, computes ω_t^{delay} per tenant over the last K rounds, and passes it to the optimizer. Jobs whose tenant is waiting above average receive $\omega_j^{\text{delay}} > 1$, so the solver naturally prefers placing them. Fairness improves as a side effect of weighted maximization. Variance of $\bar{W}_t$ across tenants (or Gini coefficient) is reported as a simulation KPI — not minimized inside the LP.

Unplaced-Job Feedback $\rightarrow \bar{W}_t$ Bump. When a job fails to be placed in a round, the Cluster Manager records a synthetic wait event of `BATCH_DURATION_SEC` (60 simulated seconds) in that tenant’s $\bar{W}_t$ rolling window. This raises ω_t^{delay} in the next solve call, ensuring persistently unplaced tenants receive priority in subsequent intervals.

6 Optimization Objective

Maximize the **Placement Score** Z :

$$\max \quad Z = \sum_{j \in J} \sum_{n \in N} \omega_j^{\text{delay}} \cdot \hat{P}_j^{\text{mem}} \cdot \omega_n^{\text{utilize}} \cdot \sigma_n^{\text{consolid}} \cdot x_{jn}$$

Note on ω_j^{delay} : $\omega_j^{\text{delay}} = \omega_{t(j)}^{\text{delay}}$, the delay weight of the tenant that owns job j . Writing ω_j^{delay} (per-job subscript j) rather than ω_t^{delay} (per-tenant subscript t) is the convention used throughout this model because the objective is indexed over jobs. Appendix C defines ω_t^{delay} and the equality follows directly: $\omega_j^{\text{delay}} = \omega_t^{\text{delay}}|_{t=t(j)}$.

Each placed job contributes its maximum predicted memory $\hat{P}_j^{\text{mem}}$, scaled by three multiplicative weights:

- $\omega_j^{\text{delay}} \geq 1$ — tenant delay weight: jobs from long-waiting tenants get higher weight, so the solver prefers placing them.
- $\omega_n^{\text{utilize}} \in [1, 2]$ — memory consolidation: a job placed on a memory-loaded node (high $U_n^{\text{mem}}/M_n^{\text{cap}}$) contributes more, consolidating workloads onto fewer machines.
- $\sigma_n^{\text{consolid}} \in \{1, \dots, |N|\}$ — fixed node bias: node 0 has the highest weight, ensuring consolidation even at the first batch when all $\omega_n^{\text{utilize}} = 1$ (no prior load history).

7 Constraints

C1 — One node per job. Each job is assigned to at most one node per scheduling round:

$$\sum_{n \in N} x_{jn} \leq 1 \quad \forall j \in J$$

≤ 1 (not $= 1$): a job may remain unscheduled if no node has sufficient capacity. Unscheduled jobs return to the queue, accumulating wait time that raises their ω_j^{delay} in subsequent rounds.

C2 — Memory capacity per node. Total maximum predicted memory of jobs assigned to node n may not exceed its effective remaining capacity:

$$\sum_{j \in J} \hat{P}_j^{\text{mem}} \cdot x_{jn} \leq M_n^{\text{eff}} \quad \forall n \in N$$

This is a hard constraint. M_n^{eff} incorporates the OS tax, safety threshold buffer, current usage, and SLA-driven scaling. The model uses the predicted maximum memory $\hat{P}_j^{\text{mem}}$ rather than the tenant's declared request (overcommitment). An SLA violation occurs only if actual runtime memory exceeds even the predicted maximum — indicating behaviour outside the training distribution.

C3 — Binary domain.

$$x_{jn} \in \{0, 1\} \quad \forall j \in J, n \in N$$

This makes the problem a Mixed-Integer Linear Program (MILP). For large instances, $x_{jn} \in [0, 1]$ (LP relaxation with rounding) is an acceptable approximation.

C4 — Per-pair CPU fitment.

$$x_{jn} = 0 \text{ if } \hat{P}_j^{\text{CPU}} > C_n \quad \forall j \in J, n \in N$$

A job may not be placed on a node whose total core count is less than the job's P95 predicted CPU peak. CPU cores are time-shared — the kernel handles overcommit via throttling. This check blocks only the genuinely infeasible case where a job's peak CPU demand exceeds the node's hardware ceiling.

Plan-Ahead Access Control (not a constraint in the LP). Machine access control is enforced *before* the solver is called. The Cluster Manager pre-filters the node list N to only those machines assigned to the current tenant group by the plan-ahead model output. The solver therefore receives only authorised machines and needs no explicit access constraint.

This replaces the former C5 constraint ($x_{jn} = 0$ if $n \notin A_\tau^{(t(j))}$), which was removed because pre-filtering achieves identical results with zero LP overhead and no interaction with C4.

8 Complete Model

$$\max \quad Z = \sum_{j \in J} \sum_{n \in N} \omega_j^{\text{delay}} \cdot \hat{P}_j^{\text{mem}} \cdot \omega_n^{\text{utilize}} \cdot \sigma_n^{\text{consolid}} \cdot x_{jn}$$

Subject to:

$$\begin{aligned} \text{C1:} \quad & \sum_{n \in N} x_{jn} \leq 1 & \forall j \in J \\ \text{C2:} \quad & \sum_{j \in J} \hat{P}_j^{\text{mem}} \cdot x_{jn} \leq M_n^{\text{eff}} & \forall n \in N \\ \text{C3:} \quad & x_{jn} \in \{0, 1\} & \forall j \in J, n \in N \\ \text{C4:} \quad & x_{jn} = 0 \text{ if } \hat{P}_j^{\text{CPU}} > C_n & \forall j \in J, n \in N \end{aligned}$$

Note: Plan-ahead access control (formerly C5) is enforced by the Cluster Manager pre-filtering N to authorised machines before the solve call. No constraint is added to the LP.

Where:

$$\begin{aligned}
M_n^{\text{cap}} &= M_n - M_n^{\text{tax}} - M_n^\theta \\
M_n^{\text{avail}} &= M_n^{\text{cap}} - U_n^{\text{mem}} \\
M_n^{\text{eff}} &= \max\left(0, M_n^{\text{avail}} \cdot (1 - \bar{v}_n^{\text{SLA}})\right) \\
\sigma_n^{\text{consolid}} &= |N| - n, \quad n = 0, 1, \dots, |N| - 1
\end{aligned}$$

Input	Source	Updated
J, N (pre-filtered)	Cluster Manager	Each group per round
$U_n^{\text{mem}}, \bar{v}_n^{\text{SLA}}, \omega_t^{\text{delay}}$	Cluster monitor	Each round (K -window)
$\hat{P}_j^{\text{mem}}, \hat{P}_j^{\text{CPU}}$	Prediction layer	Each round
$K, M_n^{\text{tax}}, M_n^\theta, \sigma_n^{\text{consolid}}$	Configuration	Constant

A How $\bar{v}_n^{\text{SLA}}$ is Computed

Additional symbols:

Symbol	Meaning
K	Rolling window length — number of past scheduling rounds included in the violation rate. Configuration scalar
k	Summation index, $k = 0, 1, \dots, K-1$. $k = 0$ is the most recent round; $k = K-1$ is the oldest
$\mathbf{1}[\cdot]$	Indicator function: 1 if the condition holds, 0 otherwise

At scheduling round r :

$$\bar{v}_n^{\text{SLA}} = \frac{1}{K} \sum_{k=0}^{K-1} \mathbf{1}[U_n^{\text{mem}} > M_n^{\text{cap}} \text{ at round } r - k]$$

$\bar{v}_n^{\text{SLA}}$ is the fraction of the last K rounds in which actual memory on node n exceeded M_n^{cap} (the capacity ceiling after OS tax and safety threshold buffer). When $\bar{v}_n^{\text{SLA}}$ rises, M_n^{eff} shrinks, throttling new job placements and reducing the risk of OOM kills. It is passed to the optimizer as an input each round by the cluster monitor.

Implementation note: Each node maintains two separate histories. `overflow_history` records rounds where $U_n^{\text{mem}} > M_n^{\text{cap}}$ (schedulable ceiling exceeded); this is what feeds $\bar{v}_n^{\text{SLA}}$. A separate `violation_history` records the stricter condition $U_n^{\text{mem}} > M_n$ (physical RAM exceeded) and is used for monitoring and reporting only — it does not enter the LP.

B How $\omega_n^{\text{utilize}}$ is Computed

$$\omega_n^{\text{utilize}} = 1 + \min\left(1, \max\left(0, \frac{U_n^{\text{mem}}}{\max(1, M_n^{\text{cap}})}\right)\right)$$

The denominator is M_n^{cap} (schedulable capacity after OS tax and safety threshold), so the weight reflects how loaded the node is relative to the memory actually available for tenant jobs. When a node is fully packed up to M_n^{cap} , the weight reaches 2.

- $\omega_n^{\text{utilize}} = 1$: node is idle (no consolidation boost)
- $\omega_n^{\text{utilize}} = 2$: node is fully packed to M_n^{cap} (maximum consolidation boost)
- $1 < \omega_n^{\text{utilize}} < 2$: partial load (real-valued and continuous)

C How ω_t^{delay} is Computed

ω_t^{delay} is the scheduling delay weight for tenant t , computed by the cluster monitor each round and inherited by every job j where $t(j) = t$ (i.e. $\omega_j^{\text{delay}} = \omega_t^{\text{delay}}|_{t=t(j)}$). Both $\bar{W}_t$ and $\bar{W}$ are computed over the last K scheduling rounds (same rolling window as $\bar{v}_n^{\text{SLA}}$), so the weight reflects *recent* fairness rather than all-time history.

Additional symbols:

Symbol	Meaning
$\bar{W}_t$	Average scheduling delay for tenant t — mean wait time from job submission to job assignment (in seconds), over the last K scheduling rounds. Both actual placed-job wait and synthetic unplaced-job wait (BATCH_DURATION_SEC per unplaced interval) are recorded
$\bar{W}$	Cluster-wide average scheduling delay over the last K rounds: $\bar{W} = \frac{1}{ T } \sum_{t \in T} \bar{W}_t$

$$\omega_t^{\text{delay}} = 1 + \max\left(0, \frac{\bar{W}_t - \bar{W}}{\max(1, \bar{W})}\right)$$

$$\omega_j^{\text{delay}} = \omega_t^{\text{delay}}|_{t=t(j)}$$

The weight is built from the excess wait above the cluster average. Adding 1 ensures the minimum is 1 (no unfairness = no extra weight). There is no stated maximum: a tenant waiting far above average receives $\omega_t^{\text{delay}} \gg 1$, giving their jobs proportionally higher priority.

- $\omega_j^{\text{delay}} = 1$ when $\bar{W}_{t(j)} \leq \bar{W}$ — no boost
- $\omega_j^{\text{delay}} > 1$ when $\bar{W}_{t(j)} > \bar{W}$ — boost grows proportionally with excess wait (real-valued, no maximum)